

# Vikas Ganjure — Technical Project Interview Guide

Architectural walkthroughs, core technical decisions, real-world trade-offs, and 50+ targeted interview questions with STAR answers for Java Backend, Spring Boot, Databases, and Full Stack roles.

 Vikas Ganjure    B.E. Computer Science (CGPA: 8.16)    Pune, Maharashtra

 Java & Backend Developer

## 1. Strategic Framework for Project Questions in Interviews

### How to Answer: "Tell me about your project"

Never start with line-by-line code. Use the **Problem-Solution-Impact** formula in 90 seconds:

1. **Context & Business Problem (20s):** Why did this project need to exist? What was broken or inefficient?
2. **Technical Architecture & Your Role (40s):** What stack did you choose and why? What was your specific architectural contribution?
3. **Complex Engineering Challenge (20s):** What tricky bug, concurrency issue, or security hurdle did you solve?
4. **Measurable Outcome (10s):** Sub-second query times, zero unauthorized breaches, clean REST contract.

## 2. Project 1: Auth & Role-Based Access Control (RBAC) System

### 30-Second Interview Pitch:

"I built an enterprise authentication and authorization microservice using Java, Spring Boot, Spring Security, and JWT. It provides stateless token-based authentication with granular role-based authorization (USER and ADMIN) and BCrypt password encryption. I designed custom Spring Security filter chains, stateless session management, and connected a MySQL database alongside a React TypeScript dashboard."

### Architecture & Request Lifecycle

Here is the complete step-by-step lifecycle of an incoming API request:

1. **Incoming HTTP Request:** Client sends request with header `Authorization: Bearer <token>`.
2. **Custom Filter ( `JwtAuthenticationFilter` ):** Intercepts request via `OncePerRequestFilter`.
3. **Token Validation:** Parses token, checks signature against secret key, checks expiration timestamp.
4. **Claims Extraction:** Extracts username and roles/authorities (e.g., `ROLE_ADMIN`, `ROLE_USER`).
5. **SecurityContext Injection:** Instantiates `UsernamePasswordAuthenticationToken` and injects it into `SecurityContextHolder.getContext().setAuthentication(...)`.
6. **Controller Access:** Controller methods protected by `@PreAuthorize("hasRole('ADMIN')")` evaluate the `SecurityContext`.
7. **Response:** If valid, returns JSON payload. If invalid/expired, returns HTTP 401 Unauthorized or 403 Forbidden.

## Top Technical Interview Questions & Answers for Project 1:

### Q: Why did you choose JWT over traditional Server-Side Sessions?

**Core Concept** "Traditional sessions store user state in server memory (`HttpSession`) and rely on a `JSESSIONID` cookie. In a distributed microservices or multi-instance architecture, this requires session replication or sticky sessions on the load balancer, which introduces overhead.

JWT is completely **stateless**: the user identity, roles, and expiration are cryptographically signed inside the token payload itself. Any backend node with the secret key can verify the token independently without querying a shared session store on every single HTTP request, making horizontal scaling seamless."

### Q: What is the anatomy of a JWT token and how is it secured?

**Security Deep-Dive** "A JWT consists of three parts separated by dots ( . ):

- **Header:** Specifies the algorithm (e.g., HS256) and token type (JWT).
- **Payload (Claims):** Contains registered claims ( `sub` , `iat` , `exp` ) and custom claims (e.g., `roles: ['ROLE_ADMIN']` ).
- **Signature:** Generated by hashing `Base64Url(Header) + "." + Base64Url(Payload)` using the server's private secret key.

Crucially, the payload is *Base64Url encoded*, not encrypted, meaning sensitive information like raw passwords should never be placed in claims. The security comes from the signature: if a malicious client tampers with their role from USER to ADMIN, the signature verification fails instantly."

### Q: How does BCrypt work and why is it preferred over SHA-256 or MD5?

**Cryptography** "MD5 and SHA-256 are fast, general-purpose cryptographic hash functions designed for throughput. Because GPUs can calculate billions of SHA-256 hashes per second, they are vulnerable to brute-force and rainbow table attacks.

BCrypt is an adaptive key-derivation function based on the Blowfish cipher. It incorporates two critical defenses: 1) **Automatic Salt:** Generates a random 128-bit salt stored within the hash string, preventing rainbow table lookups. 2) **Work Factor (Cost):** A configurable log-rounds parameter (e.g., 10 or 12) that intentionally slows down hashing calculations (taking ~100ms per attempt), making brute-force attacks computationally infeasible."

### Q: How did you resolve CORS issues between your React frontend and Spring Boot backend?

**Integration** "Because React runs on port 3000 and Spring Boot runs on port 8080 (different origins), the browser blocks cross-origin requests by sending an `OPTIONS` preflight request. I resolved this at the Spring Security level by configuring a `CorsConfigurationSource` bean in my `SecurityFilterChain` :

- Allowed origins: `http://localhost:3000`
- Allowed methods: `GET, POST, PUT, DELETE, OPTIONS`
- Allowed headers: `Authorization, Content-Type, Accept`
- Allow credentials: `true`

This ensures preflight requests pass through before authentication filters."

**Q: What was the most challenging technical hurdle in this project? (STAR Answer)**

**STAR Story** **Situation:** During initial integration, public endpoints like `/api/auth/register` and `/api/auth/login` were intermittently returning 403 Forbidden instead of generating tokens.

**Task:** Diagnose why unauthenticated requests were blocked before hitting the controller.

**Action:** I inspected the Spring Security filter order. My custom `JwtAuthenticationFilter` was throwing an unhandled exception when checking for the `Authorization` header on endpoints where no token existed yet. I updated the filter logic to verify if the header exists and starts with `Bearer` ; if not, it calls `filterChain.doFilter(request, response)` immediately to let Spring Security's `permitAll()` configuration handle public routes.

**Result:** Clean authentication flow with zero false 403 errors and robust filter isolation.

### 3. Project 2: Restaurant Management & Automated Billing System

 **30-Second Interview Pitch:**

"I developed a Java and MySQL restaurant management software system that streamlines food menu management, real-time table order tracking, and automated bill generation. The backend utilizes pure Core Java and JDBC with connection pooling for high-throughput relational data operations, transaction management for billing integrity, and a normalized MySQL schema."

**Database Schema Architecture**

| Table Name    | Primary Key     | Foreign Keys       | Core Purpose                                                                   |
|---------------|-----------------|--------------------|--------------------------------------------------------------------------------|
| menu_items    | item_id         | None               | Item name, category (Starter, Main, Drink), price, availability status         |
| orders        | order_id        | table_id           | Table number, order timestamp, order status (Open, Completed, Cancelled)       |
| order_details | order_detail_id | order_id , item_id | Item quantity, item price at time of order (prevents retroactive bill changes) |
| bills         | bill_id         | order_id           | Subtotal, tax amount (CGST/SGST), discount, final total, payment mode          |

### Top Technical Interview Questions & Answers for Project 2:

**Q: Why did you use PreparedStatement instead of Statement in JDBC?**

JDBC Fundamentals "There are two critical reasons:

- 1. **SQL Injection Prevention:** Statement uses raw string concatenation, allowing attackers to input malicious SQL fragments (e.g., ' OR '1'='1 ). PreparedStatement sends the parameterized query template to the database engine first for pre-compilation. User inputs are treated strictly as literal data values, never as executable SQL commands.
- 2. **Performance & Pre-compilation:** The database compiles and optimizes the execution plan for the query template once. When executed multiple times with different parameters, the database reuses the cached plan, saving compilation and parsing overhead."

### Q: How did you implement Database Transactions for automated billing?

**ACID Transactions** "Generating a bill involves multiple operations that must succeed or fail as a single atomic unit: 1) Calculating the final bill total, 2) Inserting a record into the `bills` table, and 3) Updating the order status in `orders` from 'Open' to 'Completed'.

In JDBC, I disabled auto-commit:

```
connection.setAutoCommit(false);
try {
    // 1. Insert into bills
    // 2. Update order status
    connection.commit(); // Commit transaction
} catch (SQLException e) {
    connection.rollback(); // Rollback if any step fails
    throw e;
} finally {
    connection.setAutoCommit(true);
}
```

This guarantees the ACID property of **Atomicity**: you never end up with a bill generated without the order being closed, or an order closed without a bill."

### Q: How did you prevent Database Connection leaks?

**Resource Management** "Database connections are scarce operating system resources (TCP sockets). In Java 7+, I leveraged the **try-with-resources** statement on all `Connection`, `PreparedStatement`, and `ResultSet` objects. Since these interfaces implement `AutoCloseable`, Java guarantees they are closed automatically in reverse order when the block exits, even if an unexpected runtime exception occurs."

### Q: Explain the Database Normalization applied in your schema.

**Database Theory** "The schema is designed up to **3rd Normal Form (3NF)**:

- **1NF**: Every column contains atomic (indivisible) values. Multiple items ordered are stored as separate rows in `order_details`, not comma-separated strings.
- **2NF**: Achieved 1NF, and all non-key attributes are fully functionally dependent on the entire primary key (no partial dependencies).
- **3NF**: Achieved 2NF, and eliminated transitive dependencies (e.g., item price is derived from `item_id` in `menu_items`, not duplicated into `orders`).

Note on intentional historical denormalization: in `order_details`, I saved the `unit_price` at purchase time so future menu price changes do not retrospectively alter past receipts."

## 4. Project 3: Malicious Social Media Post Detection (ML & NLP)

### 30-Second Interview Pitch:

"I engineered a machine learning and natural language processing web application using Python, Flask, Scikit-Learn, and MySQL. It automatically classifies online social media content and news items into benign vs malicious (phishing URLs, fake news, hate speech, and spam). The backend preprocesses text using tokenization, lemmatization, and TF-IDF vectorization before serving real-time classification verdicts via a Flask REST API."

### Machine Learning & NLP Pipeline

Raw Text → Regex Cleaning (strip URLs, tags, punctuation) → Tokenization & Lowercasing → Stop-words Removal (NLTK) → WordNet Lemmatization → TF-IDF Feature Extraction → Model Classifier (Multinomial Naive Bayes / Logistic Regression) → Confidence Score (0.0 - 1.0)

## Top Technical Interview Questions & Answers for Project 3:

### Q: What is TF-IDF and why is it better than simple Word Count (Bag of Words)?

NLP Fundamentals

"**Bag of Words (CountVectorizer)** simply counts how many times each word appears. Words like 'the', 'is', or 'this' appear frequently across all documents, drowning out distinguishing keywords."

**TF-IDF (Term Frequency - Inverse Document Frequency)** solves this:

- **TF (Term Frequency):** Measures how frequently a word appears in the specific post.
- **IDF (Inverse Document Frequency):** Penalizes words that appear commonly across the entire training corpus:  $\log(N / df)$ .

If a rare word like 'phishing', 'lottery', or 'urgent' appears frequently in a single post, its TF-IDF weight is very high, making it a strong predictive feature."

### Q: Why did you use Stemming vs Lemmatization?

NLP Preprocessing

"**Stemming** uses heuristic rule-based chopping of word suffixes (e.g., Porter Stemmer converts 'studies' to 'studi' and 'running' to 'run'). It is fast but produces non-dictionary words."

**Lemmatization** utilizes a morphological vocabulary database (like WordNet) and considers the context (Part of Speech) to reduce a word to its genuine dictionary lemma (e.g., 'better' becomes 'good', 'meeting' as a verb becomes 'meet'). Because semantics matter for toxic/malicious detection, Lemmatization delivered superior classification accuracy."

### Q: Why is Accuracy a misleading metric for spam and malicious detection? What did you use instead?

#### Model Evaluation

"In real-world data, malicious posts are rare (class imbalance: e.g., 95% benign, 5% malicious). A naive model that predicts 'benign' 100% of the time achieves 95% accuracy while failing its sole business purpose: catching malicious content.

Instead, I evaluated the model using:

- **Precision:** Out of all posts flagged as malicious, how many actually were? (Crucial to avoid falsely blocking legitimate users).
- **Recall (Sensitivity):** Out of all actual malicious posts, how many did the model catch?
- **F1-Score:** The harmonic mean of Precision and Recall:  $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$ .

F1-score provided the true benchmark for balanced performance."

### Q: How was the trained model served in production using Flask?

#### MLOps & Deployment

"Training a model is computationally intensive, so inference must be decoupled from training. Once the model and the TF-IDF vectorizer were trained, I serialized them to disk as binary artifacts using `joblib` / `pickle`. When the Flask application starts up, it loads the artifacts into memory once in the application context. When a client POSTs text to `/api/detect`, the text is transformed by the in-memory vectorizer and passed to `model.predict_proba()`, returning inference in less than 50 milliseconds without retraining."

## 5. Project 4: High-Performance Developer Portfolio (React 18 & TypeScript)

### 30-Second Interview Pitch:

"I engineered an ultra-responsive engineering portfolio using React 18, strict TypeScript, Tailwind CSS, and Vite. It features a persistent Dark/Light theme switcher, responsive glassmorphism layouts, dynamic typewriter animations, an interactive developer terminal code card, and a resilient multi-tier EmailJS contact form with instant Gmail web client fallback."

### Top Technical Interview Questions & Answers for Project 4:

#### Q: Why did you migrate from Plain HTML/JS to React + TypeScript?

Frontend Architecture

"1. **Component Reusability:** Plain HTML involved repeated markup across cards and sections. In React, UI components ( `Certifications` , `Projects` , `ThemeToggle` ) are modular, testable, and maintainable.

2. **Type Safety:** TypeScript eliminates entire classes of runtime errors. Every project, certification, and form input is typed with strict interfaces, making refactoring safe.

3. **State Management & Reactivity:** Managing theme toggles, active scroll spy, mobile drawer state, and form states with vanilla DOM manipulation requires messy event listeners and manual DOM queries. React's declarative state model cleanly separates state from presentation."

#### Q: How does React's Virtual DOM and Reconciliation work?

React Internals

"Direct browser DOM manipulation is computationally expensive because it triggers browser layout recalculations and repaints. React maintains a lightweight JavaScript representation of the DOM in memory called the **Virtual DOM**. When state changes: 1) React creates a new Virtual DOM tree. 2) It runs the **Reconciliation algorithm (Diffing algorithm)** comparing the new tree with the previous one. 3) It batches and applies only the exact modified nodes to the real DOM (called commit phase), ensuring maximum frame rate and UI responsiveness."

#### Q: How does your Theme Switcher work and persist?

State & CSS Architecture

"I built a custom `ThemeContext` utilizing React Context API and `localStorage` . When the app loads: 1) It checks `localStorage.getItem('portfolio-theme')` . 2) If not set, it detects the user's OS preference using `window.matchMedia('(prefers-color-scheme: dark)')` . 3) It toggles the `dark` class on `document.documentElement` . Tailwind CSS uses `darkMode: 'class'` , so any utility with `dark:` (e.g. `dark:bg-zinc-950 dark:text-zinc-100` ) instantly and smoothly transitions without page reloads."

## 6. High-Impact Questions Every Interviewer Asks

---

**Q: "If your RBAC system needs to handle 100,000 requests per second tomorrow, what would break first and how would you scale it?"**

System Design / Scalability

"1. **Database Bottleneck:** The relational MySQL instance would saturate on write/read IOPS. I would introduce **Redis** as an in-memory cache for user permissions and token blacklists, reducing database read load by over 90%.

2. **Read Replicas:** Deploy MySQL read replicas with master-slave replication to offload read traffic.

3. **Stateless Horizontal Scaling:** Since JWT verification is stateless, we can deploy multiple Spring Boot container instances behind an Nginx/AWS ALB load balancer.

4. **Rate Limiting:** Implement distributed token-bucket rate limiting (Bucket4j or Redis) to protect endpoints from DDoS or brute-force abuse."

**Q: "Tell me about a time you made a technical mistake and how you recovered from it."**

Behavioral / Maturity

"In my Restaurant Management System, I initially stored order totals directly in the order table and updated it dynamically. However, when menu item prices changed later, past bills and sales reports displayed inaccurate recalculated amounts.

I realized this was a flaw in temporal data modeling. I refactored the schema by capturing the historical

`unit_price` directly inside `order_details` at the time of order creation. This taught me the critical distinction between transactional audit data (which must remain immutable) and live catalog master data."

### 💡 Final Interview Day Pro-Tip:

Always be honest about what you know and how you learn. If asked about a technology you haven't used (e.g. Kafka, Kubernetes), say: *"I haven't deployed Kafka in production yet, but I understand it's an event-streaming platform using distributed append-only commit logs. In my Spring Boot project, I would use it to asynchronously process billing receipts or audit logs without blocking the user's HTTP thread."* This demonstrates architectural thinking rather than just memorized syntax!